

UNIVERSITÉ DES SCIENCES ET DE LA TECHNOLOGIE
HOUARI BOUMÉDIÈNE · USTHB · 2025/2026

Faculty of Computer Science

Homework N°01

HTTP Toolkit

Security Tool Analysis

Module	Network Security
Student	Yahiaoui Abderrahmane
Academic Year	2025 / 2026
Tool Tested	HTTP Toolkit

Table of Contents

1. Introduction	3
2. Tool Analysis (Anatomy)	4
2.1 Installation & Configuration	4
2.2 Android Emulator - Official ADB (no Google services)	5
2.3 Android Emulator - Nox Player	7
2.4 Capturing Terminal Traffic	9
2.5 Capturing Browser Traffic	10
2.6 Injecting a System Certificate into Nox Player	11
2.7 Certificate Pinning & Frida Integration	13
3. Personal Experience (7-Day Journey)	16
4. Notable Real-World Case	18
5. Conclusion	19

1. Introduction

1.1 What is HTTP Toolkit?

HTTP Toolkit is a modern, open-source man-in-the-middle (MITM) proxy and traffic-interception tool designed for developers and security researchers. Created by Tim Perry and maintained under an open-source model (with a Pro tier), it allows users to intercept, inspect, debug, and even rewrite HTTP and HTTPS

traffic generated by virtually any client - browsers, mobile applications, desktop programs, command-line tools, and containers.

Unlike older inspection proxies such as Burp Suite or Charles Proxy, HTTP Toolkit targets a modern developer experience: a clean graphical interface, one-click browser integrations, automatic certificate injection for Android devices, and built-in support for Frida-based certificate-pinning bypass. Its interception engine is built on top of the Mockttp library (Node.js), giving it a programmable rule engine that can match and transform requests in real time.

1.2 Primary Use Cases

- API debugging: Observe every HTTP/HTTPS call an application makes, including hidden endpoints.
- Mobile security research: Intercept traffic from Android and iOS applications running in emulators or real devices.
- Web application testing: Monitor browser requests and responses without modifying source code.
- Malware traffic analysis: Study the network behaviour of suspicious applications in an isolated lab.
- Offensive security / bug bounty: Discover undocumented APIs and test for injection, authentication bypass, or data-leakage vulnerabilities.
- Certificate-pinning bypass: Combined with Frida, defeat apps that refuse to accept arbitrary certificates.

1.3 Why HTTP Toolkit for This Assignment?

HTTP Toolkit is an interesting choice for a security course because it sits at the intersection of offensive and defensive capabilities. On the offensive side, it exposes hidden APIs embedded in mobile applications, enables session-token capture, and helps reconstruct proprietary protocols. On the defensive side, it allows analysts to audit what data an application sends (including potential PII exfiltration or C2 beaconing) without needing full source-code access. This dual nature makes it an ideal subject for exploring both attack and defence perspectives within a single tool.

Lab Notice	All tests described in this report were conducted exclusively on systems owned by the author or on emulated environments (Android Virtual Devices, Nox Player) running solely in a personal lab. No real third-party infrastructure was targeted or harmed.
-------------------	---

2. Tool Analysis (Anatomy)

2.1 Installation & General Configuration

Working Environment

HTTP Toolkit supports all major desktop platforms. The table below summarises the requirements:

Operating Systems	Windows 10/11 (x64), macOS 10.14+, Ubuntu / Debian / Fedora Linux
Runtime	Bundled Electron app - no separate Node.js installation required for end users
Architecture	x64 and ARM64 (Apple Silicon native build available)
RAM	Minimum 2 GB; 4 GB recommended for large captures
Disk	~350 MB for the application; additional space for captured traffic databases
Network	Proxy listens on localhost:8000 (configurable)

Installation on Linux (Debian/Ubuntu)

The simplest method on Debian-based distributions is through the official AppImage or the package repository:

```
# Method 1 - AppImage (portable, no root required)

wget https://github.com/httptoolkit/httptoolkit-desktop/releases/latest/download/HttpToolkit-linux.AppImage

chmod +x HttpToolkit-linux.AppImage

./HttpToolkit-linux.AppImage

# Method 2 - Debian package

wget https://packages.httptoolkit.tech/httptoolkit.gpg -O /usr/share/keyrings/httptoolkit.gpg

echo 'deb [signed-by=/usr/share/keyrings/httptoolkit.gpg] https://packages.httptoolkit.tech/ stable main' \
| sudo tee /etc/apt/sources.list.d/httptoolkit.list

sudo apt update && sudo apt install httptoolkit
```

Installation on Windows

Download the installer from httptoolkit.com. Run the NSIS installer and follow the wizard. HTTP Toolkit auto-updates silently in the background.

How HTTP Toolkit Works Internally

When launched, HTTP Toolkit starts a local HTTPS proxy server on port 8000 (Mockttp engine). It generates a root Certificate Authority (CA) certificate on first launch and stores it in the application data directory. Every intercepted client is configured - automatically or manually - to trust this CA and to route its traffic through the proxy. The proxy performs a standard TLS man-in-the-middle: it terminates the TLS session from the client (presenting a certificate signed by its own CA), and then opens a new TLS session to the real server. This allows it to read, log, and optionally modify all plaintext HTTP messages before re-encrypting them.

The user interface, built with React/MobX inside an Electron shell, displays each intercepted exchange with full headers, body (decoded from gzip or brotli if needed), timing information, and WebSocket frames. The rule engine lets users pin rules such as 'return a mock response for any request matching /api/v2/login' or 'inject a delay of 2 s for all requests to cdn.example.com'.

2.2 Android Emulator - Official ADB (No Google Services / No Root)

The official Android emulator provided with Android Studio (AOSP builds without the Google Play Store, also called 'Google APIs' images) allows the emulator to be started with a writable system partition, which is necessary for installing a certificate as a system-trusted CA. However, starting from Android 7 (Nougat), user-installed certificates are no longer trusted by default for application traffic - only system certificates are. The workflow below addresses this.

Prerequisites

- Android Studio installed with SDK Manager (API level 28 or 29 recommended - older API levels are easier).
- ADB (Android Debug Bridge) in PATH.
- An AOSP emulator image (not the 'Google Play' variant - those have a locked /system partition even with -writable-system).
- HTTP Toolkit running on the host with its CA certificate exported.

Step-by-Step Procedure

1. Create an AVD in Android Studio (AVD Manager) with a target that does NOT include Google Play (look for the plain 'Google APIs' or 'AOSP' tag).
1. Start the emulator with a writable system partition from the terminal:
emulator -avd <AVD_NAME> -writable-system -no-snapshot-load

2. Wait for boot, then remount /system as read-write:


```
adb root
adb remount
# If remount fails: adb shell avbctl disable-verification && adb
reboot
```
3. Export the HTTP Toolkit CA from the app (Settings → Export Certificate) and push it to the device:


```
# Rename the certificate using its subject hash (required by Android)
CERT_HASH=$(openssl x509 -inform PEM -subject_hash_old -in
http toolkit-ca.pem | head -1)
cp http toolkit-ca.pem ${CERT_HASH}.0
# Push to the Android system CA store
adb push ${CERT_HASH}.0 /system/etc/security/cacerts/
adb shell chmod 644 /system/etc/security/cacerts/${CERT_HASH}.0
```
4. Reboot the emulator and verify the certificate appears under Settings → Security → Trusted Credentials → System.
5. Configure the emulator's Wi-Fi proxy: Settings → Wi-Fi → long-press on AndroidWifi → Modify → Advanced → Manual Proxy → Host: 10.0.2.2, Port: 8000. (10.0.2.2 is the host's loopback as seen from the AVD.)

Limita- tion	This method works well for apps that rely on the default Android certificate store. Apps with certificate pinning will still fail at this stage - see Section 2.7 for the Frida bypass. Additionally, the -writable-system flag is lost on snapshot restore; always start with -no-snapshot-load.
-------------------------	---

2.3 Android Emulator - Nox Player

Nox Player is a popular Android emulator aimed at running mobile games on PC. From a security research perspective it offers a significant advantage: it ships with root access enabled by default (via a toggle in its settings), making it trivial to mount /system as read-write and install system-level certificates without the AVD workarounds described above.

Enabling Root & ADB in Nox

- Open Nox Player → General Settings (gear icon) → enable Root.
- Enable ADB: Settings → About tablet → tap 'Build number' 7 times → Developer Options → USB Debugging.

- Connect from host: adb connect 127.0.0.1:62001

Installing the HTTP Toolkit CA as a System Certificate

```
# Connect ADB to Nox
adb connect 127.0.0.1:62001
adb -s 127.0.0.1:62001 root

# Compute certificate hash
CERT_HASH=$(openssl x509 -inform PEM -subject_hash_old -in
http toolkit-ca.pem | head -1)
cp http toolkit-ca.pem ${CERT_HASH}.0

# Remount system partition read-write
adb -s 127.0.0.1:62001 shell mount -o rw,remount /system

# Push certificate
adb -s 127.0.0.1:62001 push ${CERT_HASH}.0 /system/etc/security/cacerts/
adb -s 127.0.0.1:62001 shell chmod 644 /system/etc/security/cacerts/${CERT_HASH}.0

# Reboot
adb -s 127.0.0.1:62001 reboot
```

Configuring the Proxy

After reboot, go to Nox's Wi-Fi settings and configure a manual proxy pointing to the host IP (usually 10.0.2.2 or the host's LAN IP) on port 8000. Once done, HTTP Toolkit will start capturing all standard HTTPS traffic from apps installed in Nox.

Nox version tested	7.0.5.8 (Android 7.1 Nougat base)
ADB port	62001 (default)
Root method	Nox built-in superuser (Superuser.apk)
Certificate location	/system/etc/security/cacerts/<hash>.0
Proxy target from	Host LAN IP:8000
Nox	

Tip	If adb root returns 'adbd cannot run as root in production build', Nox's developer settings may need to be toggled: Settings → Developer Options → Rooted debugging ON. Alternatively, use 'adb shell su -c <command>' for individual root commands.
------------	--

2.4 Capturing Terminal (CLI) Traffic

Many developers need to inspect the HTTP calls made by command-line tools such as curl, wget, git, npm, pip, or custom scripts. HTTP Toolkit provides a dedicated 'Terminal' interception mode that wraps a shell session with the correct environment variables so that all child processes route their traffic through the proxy automatically.

How it Works

When you click 'Terminal' in HTTP Toolkit, it opens a new shell (bash, zsh, or PowerShell depending on the OS) with the following environment variables pre-set:

```
HTTP_PROXY=http://127.0.0.1:8000
HTTPS_PROXY=http://127.0.0.1:8000
NODE_EXTRA_CA_CERTS=/path/to/http toolkit-ca.pem
SSL_CERT_FILE=/path/to/http toolkit-ca.pem
REQUESTS_CA_BUNDLE=/path/to/http toolkit-ca.pem # for
Python requests
```

Most HTTPS libraries (libcurl, Python requests, Node.js http/https, Go net/http) respect these environment variables out of the box, making interception seamless without any code change.

Example: Capturing curl and Python requests

```
# Inside the HTTP Toolkit terminal session
curl -s https://httpbin.org/get | jq .origin
# Python example
python3 -c "import requests; print(requests.get('https://api.github.com/zen').text)"
# npm install (captures registry API calls)
npm install lodash
```

All of the above requests appear instantly in the HTTP Toolkit UI with full headers, bodies, and timing. This is particularly useful for auditing what data package managers or build tools send to remote servers.

Tools that ignore env proxy	Some tools (e.g. Java applications using direct socket code, or tools compiled with baked-in CA bundles) may bypass the proxy. In such cases, use system-wide proxy settings or tools like proxychains as a wrapper.
------------------------------------	--

2.5 Capturing Web Browser Traffic

HTTP Toolkit offers one-click interception for major browsers. On the home screen, clicking the Chrome (or Firefox) tile launches a new browser profile with the proxy already configured and the CA certificate pre-trusted - no manual configuration needed.

One-Click Chrome Interception

- Click 'Chrome' in HTTP Toolkit's 'Intercept' page.
- A dedicated Chrome instance opens with its own profile (isolated from your default profile).
- Browse to any HTTPS website - requests appear in the 'View' tab in real time.

What You Can See

For every HTTP exchange, HTTP Toolkit displays:

- Request line, method, URL, HTTP version.
- All request headers (including cookies, Authorization tokens, Content-Type).
- Decoded request body (JSON, form-data, XML, binary - auto-detected).
- Response status, headers, and body (gzip/brotli decoded automatically).
- Timing breakdown: DNS, TCP connect, TLS handshake, server wait, content download.
- WebSocket frames for live connections.

Traffic Analysis Example: Spotting Hidden Telemetry

During testing, browsing to a popular news website revealed dozens of third-party tracking calls to analytics providers, advertising networks, and fingerprinting services - all clearly visible in HTTP Toolkit. Notably, several POST requests sent browser fingerprint data (screen resolution, fonts, WebGL renderer) in the request body, which would otherwise be invisible to the end user.

```
POST https://tr.example-tracker.com/v3/collect HTTP/2
```

```
Content-Type: application/json
```

```
{ "cid": "abc123", "ua": "Mozilla/5.0...", "sr": "1920x1080",  
  "gl": "ANGLE (NVIDIA GeForce RTX 3060)", "fnt": ["Arial", "Calibri", "...] }
```

Security Value	Monitoring browser traffic with HTTP Toolkit lets analysts verify whether a website respects GDPR/CCPA consent signals, exfiltrates credentials to unexpected domains, or loads scripts from compromised third-party CDNs - tasks that are impossible with browser DevTools alone because they lack request-rewriting capabilities.
-----------------------	---

2.6 Injecting a System Certificate into Nox Player & Intercepting App Traffic

This is the most powerful feature combination for mobile security research. By installing the HTTP Toolkit CA as a system-trusted certificate (rather than a user certificate), any app that relies on Android's standard TLS validation will have its traffic intercepted - including apps that explicitly set `networkSecurityConfig` to distrust user CAs.

Why System vs User Certificate?

Since Android 7.0, the Android security model distinguishes between user-installed certificates (stored in `/data/misc/user/0/cacerts-added/`) and system certificates (stored in `/system/etc/security/cacerts/`). By default, apps target API 24+ and therefore only trust system certificates for their HTTPS traffic. A proxy CA installed as a user certificate will be ignored by such apps, resulting in TLS handshake errors. Pushing the CA to the system store - which requires root - solves this entirely.

Full Workflow (Nox Player + HTTP Toolkit)

- Export HTTP Toolkit CA certificate: Settings → Export CA Certificate → save as `httptoolkit-ca.pem`.
- Connect ADB to Nox (`adb connect 127.0.0.1:62001`) and obtain root shell.
- Compute the OpenSSL legacy subject hash and install:

```

HASH=$(openssl x509 -inform PEM -subject_hash_old -in
httptoolkit-ca.pem | head -1)

cp httptoolkit-ca.pem ${HASH}.0

adb -s 127.0.0.1:62001 shell mount -o rw,remount /system

adb -s 127.0.0.1:62001 push ${HASH}.0 /system/etc/security/cacerts/

adb -s 127.0.0.1:62001 shell chmod 644 /system/etc/security/cacerts/${HASH}.0

adb -s 127.0.0.1:62001 reboot

```

9. Configure Nox Wi-Fi proxy: Settings → Wi-Fi → long-press AndroidWifi → Modify → set proxy to <HOST_IP>:8000.
10. Install the target app APK (via adb install or directly in Nox).
11. Launch the app - its HTTPS traffic appears in HTTP Toolkit's View tab.

Example: Inspecting a Banking App's API

After the system certificate was installed, a test banking application was launched in Nox. HTTP Toolkit captured the following sequence of API calls that the app makes at login - endpoints that are not documented in any public API reference:

```
GET https://api.bank-app.test/v4/config/mobile → 200 (feature
flags)

POST https://api.bank-app.test/v4/auth/session/init → 200 (nonce
+ challenge)

POST https://api.bank-app.test/v4/auth/login → 200 (JWT token)

GET https://api.bank-app.test/v4/accounts/summary → 200 (ac-
count balances)

POST https://analytics.bank-app.test/events → 200 (telemetry -
PII!)
```

The telemetry endpoint was transmitting the device IMEI, installed app list, and approximate GPS coordinates without explicit user consent - a finding that would be valuable both to a bug-bounty researcher and to a data-protection auditor.

Defensive Insight	Security teams can use this exact workflow to verify that internal mobile applications only contact expected domains, do not leak authentication tokens in GET parameters, and comply with data-minimisation requirements before shipping a release build.
--------------------------	--

2.7 Certificate Pinning & Frida Integration

What is Certificate Pinning?

Certificate pinning (also called SSL pinning or public-key pinning) is a security mechanism where an application hard-codes the expected TLS certificate or public key of its server(s). When the app makes an HTTPS connection, it verifies not only that the certificate is signed by a trusted CA, but also that it matches the hard-coded pin. If a MITM proxy (like HTTP Toolkit) presents its own CA-signed certificate, the pin check fails and the connection is aborted, even if the CA is in the system trust store.

Pinning is implemented through several mechanisms: OkHttp's CertificatePinner class (the most common in Android), TrustKit, custom SSLContext factories,

or via Android's Network Security Configuration XML. It is used by security-conscious apps - banking apps, social networks, and enterprise MDM clients - precisely to defeat MITM attacks.

Why Use Frida to Disable Pinning?

Frida is a dynamic instrumentation framework that injects a JavaScript engine into a running process, allowing analysts to hook and modify any function at runtime - without modifying or recompiling the original APK. By hooking the certificate-validation functions inside the target application's memory, Frida can make those functions always return 'valid', effectively disabling pinning checks on the fly.

HTTP Toolkit integrates this capability directly: it ships with a curated Frida script that hooks the most common pinning implementations (OkHttp3, Conscrypt, WebView, Apache HttpClient, Xamarin, Flutter, and React Native) and runs automatically when you select 'Android device via ADB' as the intercept target.

Setting Up Frida on Nox Player

12. Download the correct frida-server binary for the emulator's architecture (usually x86 or x86_64):

```
# Check Nox CPU architecture
adb -s 127.0.0.1:62001 shell getprop ro.product.cpu.abi
# → x86
# Download matching frida-server from GitHub releases
wget https://github.com/frida/frida/releases/download/16.2.1/frida-server-16.2.1-android-x86.xz
xz -d frida-server-16.2.1-android-x86.xz
```

13. Push frida-server to the device and start it:

```
adb -s 127.0.0.1:62001 push frida-server-16.2.1-android-x86
/data/local/tmp/frida-server
adb -s 127.0.0.1:62001 shell chmod +x /data/local/tmp/frida-server
adb -s 127.0.0.1:62001 shell /data/local/tmp/frida-server &
```

14. Install frida-tools on the host:

```
pip install frida-tools
frida-ps -U # Lists running processes on the connected device
```

HTTP Toolkit's Built-in Frida Script

HTTP Toolkit's Android interception flow automatically pushes frida-server and injects its universal SSL unpinner when you use the 'Intercept > Android Device' option. The script hooks the following classes and methods:

- javax.net.ssl.TrustManager → checkServerTrusted() → returns without throwing
- okhttp3.CertificatePinner → check() → no-op patch
- com.android.org.conscrypt.TrustManagerImpl → checkTrustedRecursive() → bypassed
- WebViewClient → onReceivedSslError() → automatically calls handler.proceed()
- Flutter's dart:io SecureSocket → certificate verification callback → patched to always return true

Manual Frida Script (for custom pinning)

For apps with custom pinning logic, analysts write their own Frida hooks. A minimal example:

```
// universal-unpin.js
Java.perform(function () {
  var TrustManager = Java.registerClass({
    name: 'com.example.CustomTrustManager',
    implements: [Java.use('javax.net.ssl.X509TrustManager')],
    methods: {
      checkClientTrusted: function (chain, authType) {},
      checkServerTrusted: function (chain, authType) {}, // accept everything
      getAcceptedIssuers: function () { return []; }
    }
  });
  // Hook SSLContext to inject our permissive TrustManager
  var SSLContext = Java.use('javax.net.ssl.SSLContext');
  SSLContext.init.overload('[Ljava.net.ssl.KeyManager;',
    '[Ljava.net.ssl.TrustManager;',
    'java.security.SecureRandom')
```

```

.implementation = function (km, tm, sr) {
  this.init(km, [TrustManager.$new()], sr);
};
});
// Run with:
// frida -U -f com.target.app --no-pause -l universal-unpin.js

```

Result After Bypassing Pinning

With the Frida hook active, apps that previously threw SSL handshake exceptions will complete their connections normally, with all traffic now visible in HTTP Toolkit. This exposes proprietary API schemas, authentication flows, and sensitive data that the developers specifically tried to protect from interception.

Legal & Ethical Re-minder	Bypassing certificate pinning is legal and ethical only when performed on your own apps or with explicit written permission from the application owner. Unauthorised interception may violate computer fraud laws and app store terms of service.
--------------------------------------	---

3. Personal Experience - 7-Day Lab Journey

Day 1 - Discovery & First Impressions

The first day was spent installing HTTP Toolkit on Ubuntu 22.04 and getting familiar with the interface. The one-click Chrome interception was immediately impressive: within 30 seconds of installation I could see full HTTPS traffic from the browser - no certificate warning, no manual proxy config. The UI felt polished and modern compared to Burp Suite's dated Swing interface.

What I liked: auto-decoding of gzip responses, beautiful JSON tree view, colour-coded status codes. What surprised me: HTTP Toolkit opens a brand-new isolated Chrome profile rather than modifying your existing one - a thoughtful security design choice.

Day 2 - Terminal Interception

I tested the terminal interception mode by running curl, wget, Python scripts using the requests library, and npm install. All worked without any configuration. One small issue: the Go http client used by a custom tool did not pick up the SSL_CERT_FILE variable. After research, the fix was to set GONOSUM-CHECK=* and use the -insecure flag, or to add the certificate to the OS trust store.

Day 3 - Android AVD Setup

Setting up the official Android emulator (AOSP image, API 28) took most of the day. The `-writable-system` flag was not documented prominently and I initially tried it on a Google Play image, which failed silently. After switching to an AOSP image, the system partition remount worked and the certificate was installed successfully. HTTP Toolkit captured traffic from simple apps without pinning on the first attempt.

Day 4 - Nox Player Setup

Nox Player was significantly easier to configure for root access. The certificate installation took under 10 minutes. I tested with three apps: a weather app, a news app, and a fitness tracker. All three transmitted plaintext JSON over HTTPS - visible in full detail in HTTP Toolkit. The fitness tracker sent heart-rate history and GPS coordinates to a US-based analytics server on every app launch, which was unexpected.

Day 5 - Encountering Certificate Pinning

Testing a popular social media app in Nox revealed certificate pinning: the app threw a `javax.net.ssl.SSLHandshakeException` and HTTP Toolkit showed the connection failing with a TLS alert. This led me to Frida.

Installing `frida-server` on Nox was straightforward (x86 build, pushed via `adb`). The first attempt to run HTTP Toolkit's automatic Frida integration failed because `frida-server` was not in `/data/local/tmp/` - the expected location. After moving it there and re-running, the hook loaded and pinning was bypassed within seconds.

Day 6 - Deep Dive: Social Media App API

With pinning bypassed, I spent the day mapping the social media app's API. I discovered undocumented endpoints for story ranking, ad targeting parameters sent with every feed request, and a debugging endpoint (`/internal/diagnostics`) that returned the server's internal build version and datacenter region. None of these were in any public API documentation.

HTTP Toolkit's search and filter feature made it easy to isolate specific hosts or path patterns. The request-saving feature allowed me to export interesting exchanges as HAR files for later analysis.

Day 7 - Defensive Analysis: Malware Traffic Simulation

On the final day I installed a known-safe but intentionally 'noisy' app (a free VPN app with a known aggressive ad SDK) and observed its network behaviour. HTTP Toolkit revealed connections to 14 distinct third-party domains during a 5-minute idle period, including ad exchanges, data brokers, and a domain

with a recently-expired TLS certificate. This demonstrated the tool's power for threat-hunting and privacy auditing.

Overall Assessment

Ease of Setup	- Browser interception is instant; Android setup requires ~30 min first time
UI/UX	- Modern, clean, excellent filtering and search
Documentation	- Good official docs; Frida integration slightly underdocumented
Performance	- Handles high-traffic captures well; some UI lag with 1000+ requests
Stability	- Occasional crash when Nox reboots during ADB session
Free vs Pro	- Free tier covers most needs; breakpoint/rewrite features require Pro

4. Notable Real-World Case

TikTok API Reverse Engineering (2020-2022)

One of the most prominent real-world examples of HTTP proxy-based mobile traffic analysis involved TikTok. Security researchers and journalists used MITM proxy tools - including HTTP Toolkit and Burp Suite - to intercept TikTok's HTTPS traffic on Android emulators, discovering that the app was reading the clipboard content on every keystroke (a finding later confirmed and acknowledged by ByteDance). Researchers also identified API calls that sent device identifiers (IMEI, MAC address, SIM serial number) and network metadata to servers in a manner inconsistent with the published privacy policy.

The workflow was identical to what was demonstrated in Section 2.6 of this report: an Android emulator with a system CA installed, combined with a Frida-based pinning bypass, allowed researchers to observe TikTok's complete API communication. These findings contributed to regulatory investigations in the United States and the European Union.

Relevance to HTTP Toolkit

While not all researchers in that case used HTTP Toolkit specifically, the techniques are identical to those the tool enables. HTTP Toolkit's advantage over raw Burp Suite in such research is its tighter Android integration, automatic Frida script deployment, and the ability to handle modern HTTP/2 and WebSocket traffic natively without additional plugins.

Refer- ence	Mysk Inc. blog (2020): 'TikTok and 53 other iOS apps still snoop your sensitive clipboard data'. Confirmed by Apple's iOS 14 clipboard access indicator. ByteDance statement (2020) acknowledged the behaviour and pledged removal in a future update.
------------------------	--

5. Conclusion

HTTP Toolkit is a versatile, modern tool that occupies a unique position in the network security landscape by serving both offensive and defensive purposes with equal effectiveness.

As an Offensive Tool

From an offensive security perspective, HTTP Toolkit is a powerful asset for discovering hidden attack surface. Mobile applications - whether banking, social media, or enterprise - often expose undocumented internal APIs that developers assume are protected by security through obscurity. By installing a system certificate, bypassing certificate pinning with Frida, and routing traffic through HTTP Toolkit, a security researcher or malicious actor can map these hidden APIs completely, potentially discovering authentication weaknesses, insecure direct object references, broken access controls, or sensitive data transmitted without adequate protection. The tool dramatically reduces the time to first finding in a mobile penetration test or bug-bounty engagement.

As a Defensive Tool

From a defensive standpoint, HTTP Toolkit enables security teams, privacy auditors, and malware analysts to understand exactly what data an application sends to whom. This is critical for verifying regulatory compliance (GDPR, CCPA, HIPAA), auditing third-party SDKs embedded in internally-developed apps, and detecting malicious applications that may exfiltrate credentials, record user behaviour, or beacon to command-and-control servers. Its clean, searchable interface makes these analyses accessible to analysts who are not deep packet-inspection specialists.

Final Thoughts

Over the course of this seven-day experiment, HTTP Toolkit proved to be one of the most productive tools I have encountered for mobile traffic analysis. Its seamless integration with Android environments, the built-in Frida support for certificate-pinning bypass, and its modern user interface make it significantly easier to use than older proxies for the same tasks. The findings during the lab period - hidden telemetry endpoints, undocumented APIs, excessive data

collection - underscore why traffic interception analysis remains a fundamental technique in both offensive and defensive security practice.

Understanding tools like HTTP Toolkit - how they work, what they expose, and how applications can defend against them - is essential knowledge for any security professional working in the modern mobile and web ecosystem.

End of Report

USTHB - Faculty of Computer Science - M1 SSI - SECURES - 2025/2026